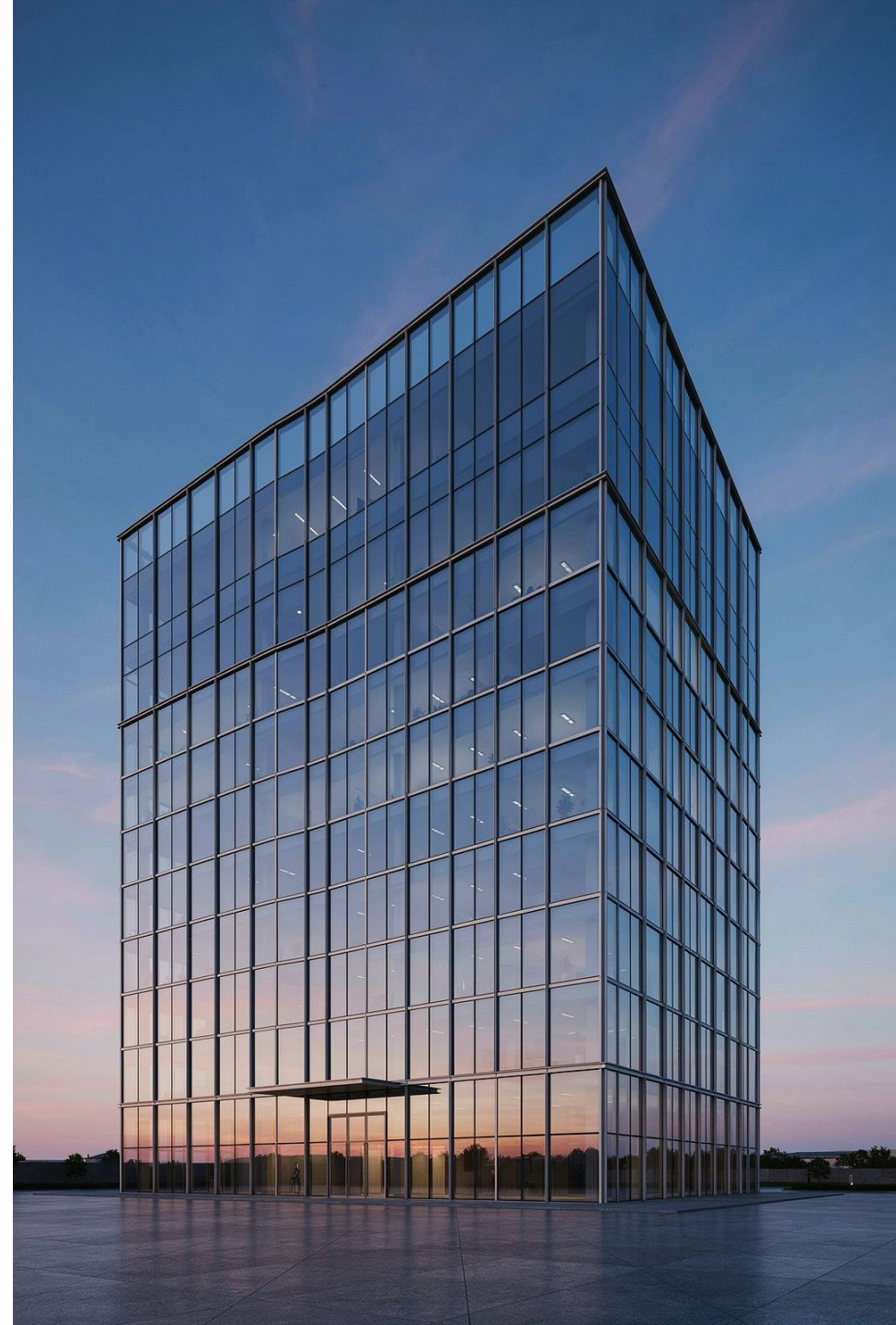


Day 2 — Threat Modeling & Cloud Security

A practical, story-driven journey through the principles, tools, and techniques that security engineers use every day to build and defend cloud-native systems.

SECURITY TRAINING · DAY 2



What We Cover Today

Six focused modules — from foundational design principles to encryption — each building on the last to form a complete security mindset.



Module 1

Security Design Principles



Module 2

Threat Modeling Fundamentals



Module 3

Threat Modeling Methodologies



Module 4

API Security



Module 5

Microservices Security



Module 6

Encryption



MODULE 1

Security Design Principles

Great security isn't bolted on after the fact — it's designed in from the start. These five principles are the architect's toolkit for building systems that resist attack by design, not by chance.

The CIA Triad — The Foundation of Everything

Every security decision ultimately protects one or more of these three properties. Think of a hospital's patient record system: records must be **Confidential** (only authorized staff can read them), **Intact** (no one can tamper with dosage data), and **Available** (nurses need access at 3 a.m., not just 9 a.m.).

Confidentiality

Only authorized users see sensitive data.
Enforced with encryption, access controls,
and least privilege.

Integrity

Data is accurate and unmodified. Enforced
with hashing, digital signatures, and audit
logs.

Availability

Systems are up when needed. Enforced with
redundancy, rate limiting, and DDoS
protection.

Least Privilege, Defense in Depth & Zero Trust

Least Privilege

Give users and services only the permissions they need — nothing more. A Lambda function that reads from S3 should never have delete rights. Real-world analogy: a hotel housekeeper has a room key, not a master key.

Defense in Depth

Layer your defenses. Even if an attacker breaches the firewall, they still face authentication, then encryption, then audit logging. Like a bank vault: outer door, inner door, guard, camera.

Zero Trust

"Never trust, always verify." In a Zero Trust model, no user or service is trusted by default — not even those inside your own network. Every request must be authenticated, authorized, and encrypted, regardless of origin.

Real-world analogy: A concert venue that checks your ticket AND your ID at every section — not just at the front gate.

Secure by Design

Security requirements are baked in during architecture and coding — not patched in after a breach. Security reviews happen at every stage of development.

MODULE 2

Threat Modeling Fundamentals

Threat modeling is the practice of thinking like an attacker — systematically identifying what could go wrong before it does. It turns security from reactive to proactive.



What Is Threat Modeling — and Why Does It Matter?

What Is It?

Threat modeling is a structured process for identifying, categorizing, and prioritizing potential threats to a system. You ask four key questions:

1. What are we building?
2. What can go wrong?
3. What are we going to do about it?
4. Did we do a good job?

A Real-World Analogy

Before opening a coffee shop, the owner asks: "Can someone rob the register? Can food spoil? Can a fire break out?" Threat modeling is that same risk conversation — for software.

Why Bother?

- Find flaws early

Fixing a design flaw costs 30× less than fixing it post-deployment.

- Prioritize risk

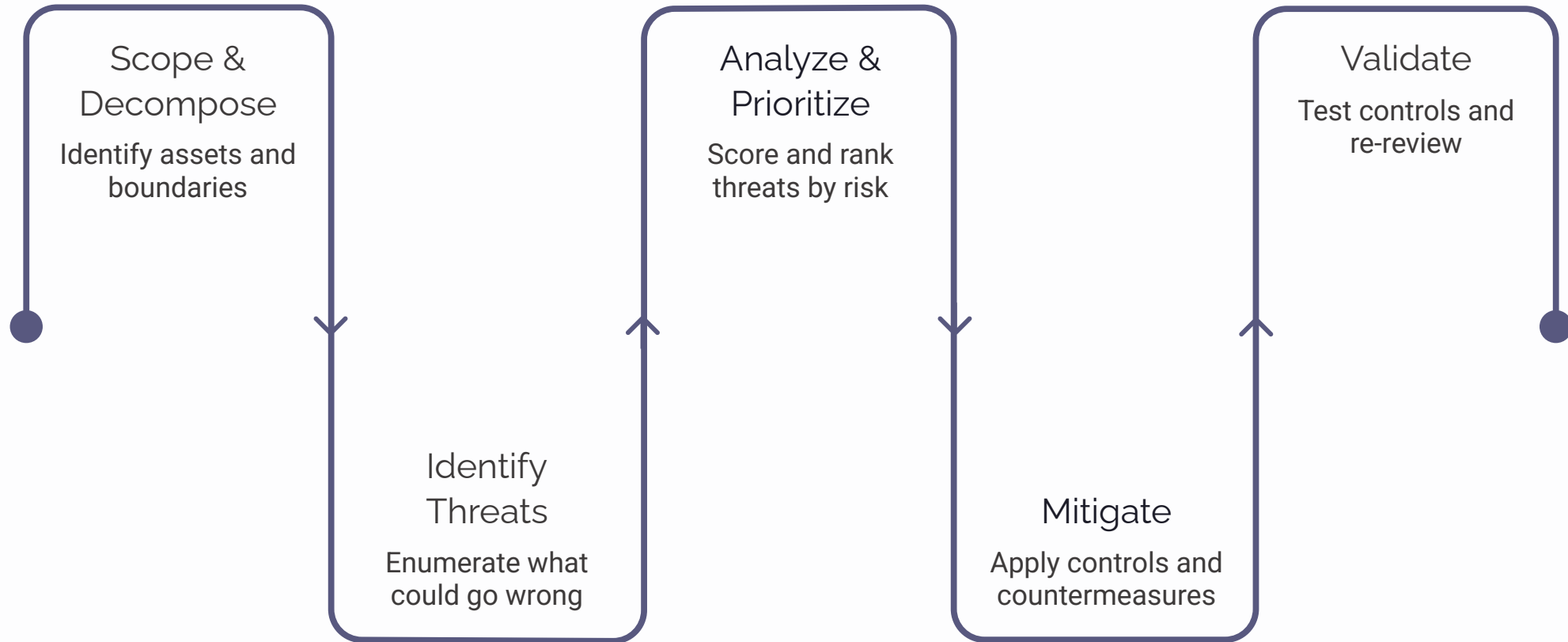
Not all threats are equal. Focus engineering effort where it matters most.

- Build shared understanding

Aligns dev, ops, and security teams on what the system must protect.

The Threat Modeling Lifecycle

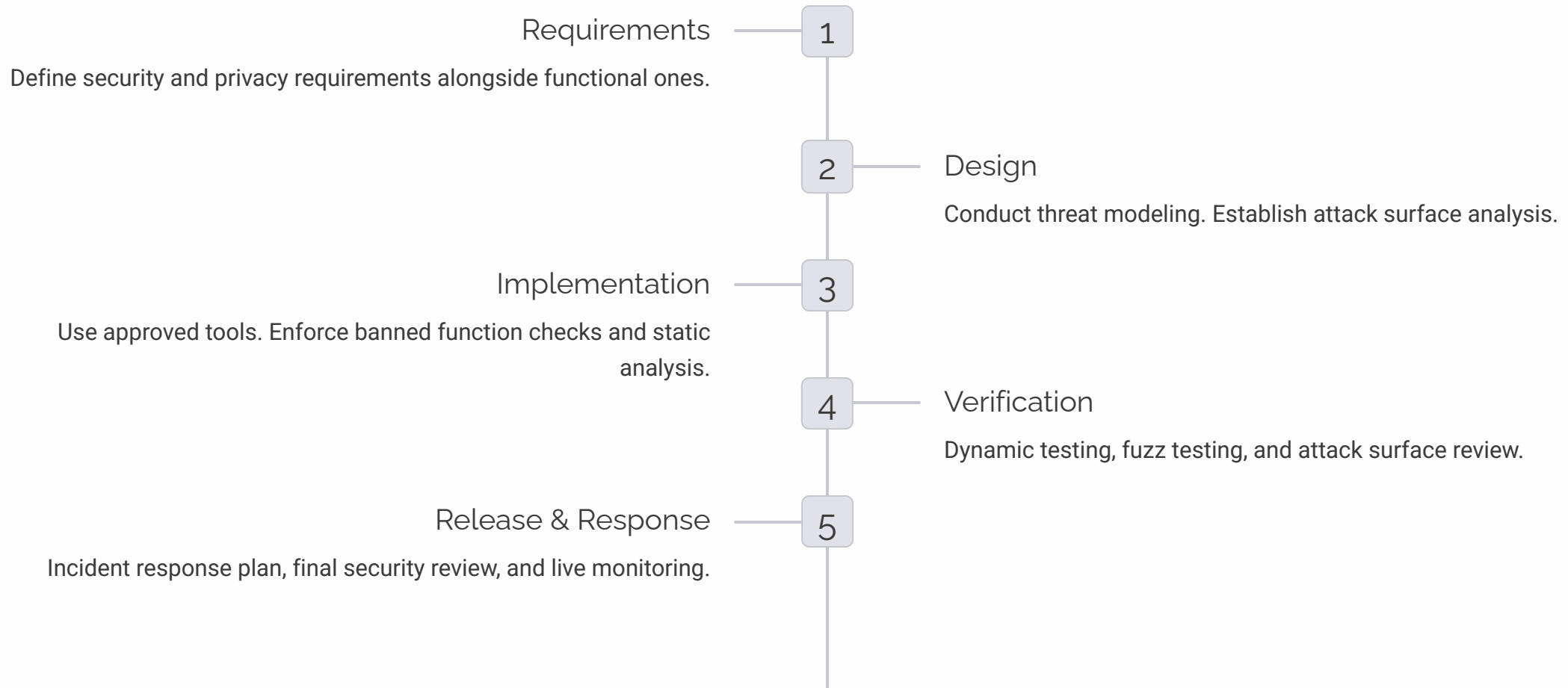
Threat modeling isn't a one-time meeting — it's an ongoing discipline embedded into your development process, revisited whenever the system changes.



Each phase feeds the next — and the cycle repeats with every new feature, integration, or architecture change.

Security Development Lifecycle (SDL)

Microsoft's SDL — now industry standard — integrates security activities at every phase of software development. The key insight: the later you find a vulnerability, the more expensive it is to fix.



MODULE 3

Threat Modeling Methodologies

Different frameworks give security teams structured lenses for identifying and rating threats. The right methodology depends on your context — from rapid threat categorization to deep risk scoring.



STRIDE — Naming the Threat

STRIDE, developed by Microsoft, gives every threat a category. When you build a data flow diagram, you walk each component through all six threat types. **Example:** A payment API could face Spoofing (fake merchant identity), Tampering (modify transaction amount), Repudiation (deny placing an order), Information Disclosure (expose card data), Denial of Service (flood the endpoint), and Elevation of Privilege (user gains admin access).

S — Spoofing

Pretending to be someone else. Mitigate with strong authentication.

T — Tampering

Modifying data in transit or at rest. Mitigate with integrity checks.

R — Repudiation

Denying an action was taken. Mitigate with audit logging.

I — Info Disclosure

Leaking confidential data. Mitigate with encryption and ACLs.

D — Denial of Service

Overwhelming a service. Mitigate with rate limiting and redundancy.

E — Elevation of Privilege

Gaining unauthorized permissions. Mitigate with least privilege.

DREAD, PASTA & Attack Trees

DREAD — Risk Scoring

DREAD scores each threat on five dimensions (1–10 each), giving a numeric risk priority:

- **Damage Potential** — How bad is the impact?
- **Reproducibility** — How easily can it be repeated?
- **Exploitability** — How easy is it to exploit?
- **Affected Users** — How many users are impacted?
- **Discoverability** — How easy is it to find?

Total score helps engineering teams triage which threats to fix first.

PASTA — Process for Attack Simulation

A risk-centric, seven-stage methodology that simulates real attacker behavior against business objectives. Best for mature security programs needing deep business impact analysis.

Attack Trees

Visual, tree-structured representations of how an attacker could reach a goal. The root node is the attacker's goal (e.g., "Steal customer PII"). Branches are the attack paths. Leaves are the individual steps. Great for brainstorming and red-team exercises.

Data Flow Diagrams & Trust Boundaries

Before you can find threats, you need to visualize your system. Data Flow Diagrams (DFDs) map how data moves through your system — and where it crosses trust boundaries.

DFD Components

External Entities

Users, partners, or third-party services that interact with your system.

Processes

Components that transform or act on data (e.g., an auth service).

Data Stores

Databases, caches, and file systems where data lives at rest.

Data Flows

Arrows showing how data moves between components.

Trust Boundaries

A trust boundary is a line in your DFD where data moves between different levels of trust. **Every time data crosses a trust boundary, it must be validated, authenticated, and authorized.**

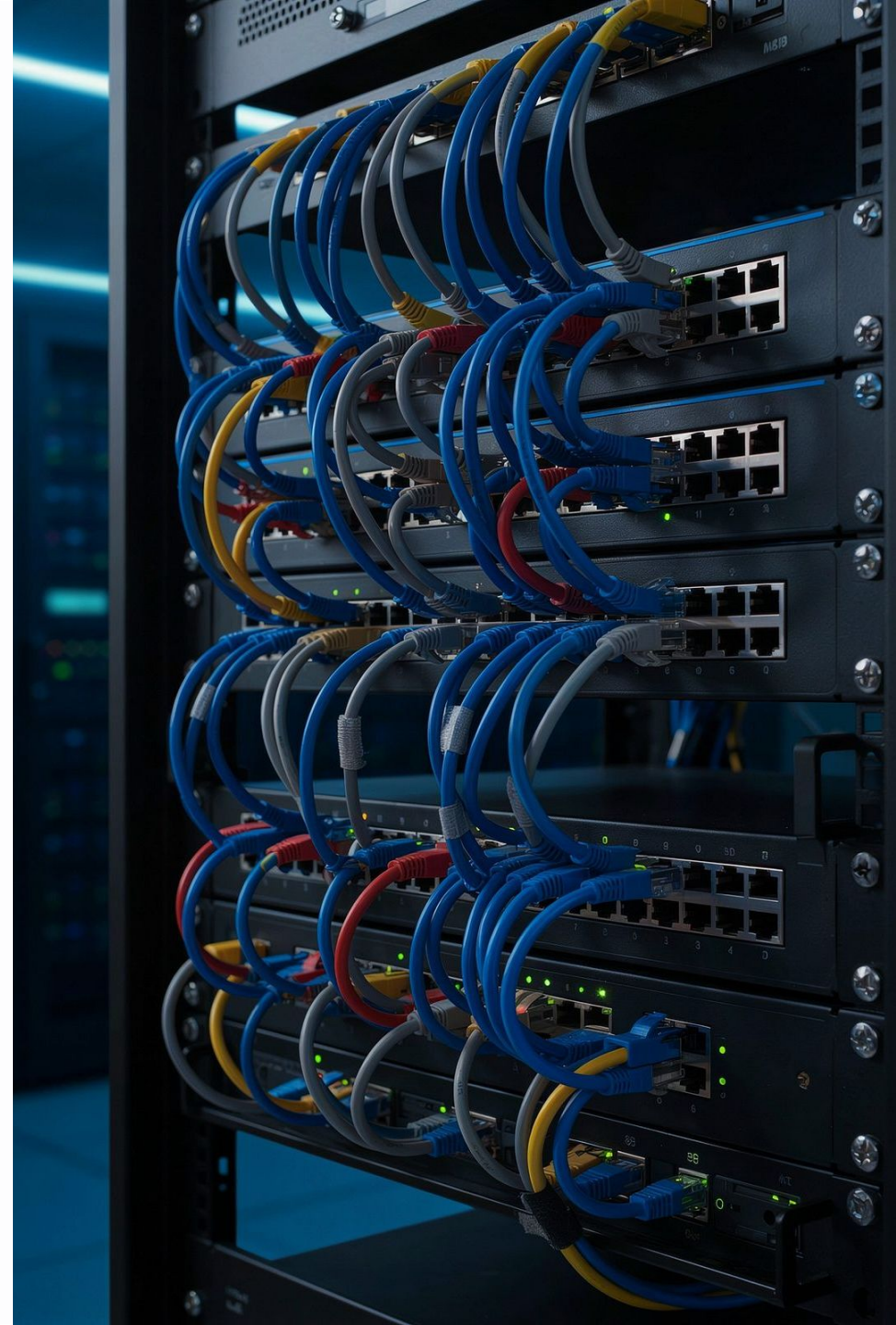
Example: A mobile app (untrusted) sending a request to your API gateway (trusted zone) crosses a trust boundary. That request must be authenticated via a token and the payload validated before it touches your backend.

📌 Trust boundaries are where most real-world breaches begin — treat every crossing as potentially hostile.

MODULE 4

API Security

APIs are the front door of modern cloud applications. Securing them — from authentication to rate limiting — is non-negotiable for any cloud-native system.



OWASP API Top 10 & Core API Security Controls

OWASP API Top 10 (Key Risks)

1. Broken Object Level Authorization (BOLA)
2. Broken Authentication
3. Broken Object Property Level Authorization
4. Unrestricted Resource Consumption
5. Broken Function Level Authorization
6. Server-Side Request Forgery (SSRF)
7. Security Misconfiguration
8. Lack of Protection from Automated Threats

Real-world example: BOLA let attackers access any user's medical records simply by changing the ID in the URL – no hacking required.

Essential API Security Controls

→ Authentication with OAuth2 & JWT

OAuth2 delegates access (e.g., "Login with Google"). JWT carries identity claims in a signed, verifiable token. Always verify the signature and expiry.

→ Authorization

Authentication proves who you are – authorization decides what you can do. Enforce at every endpoint, not just the gateway.

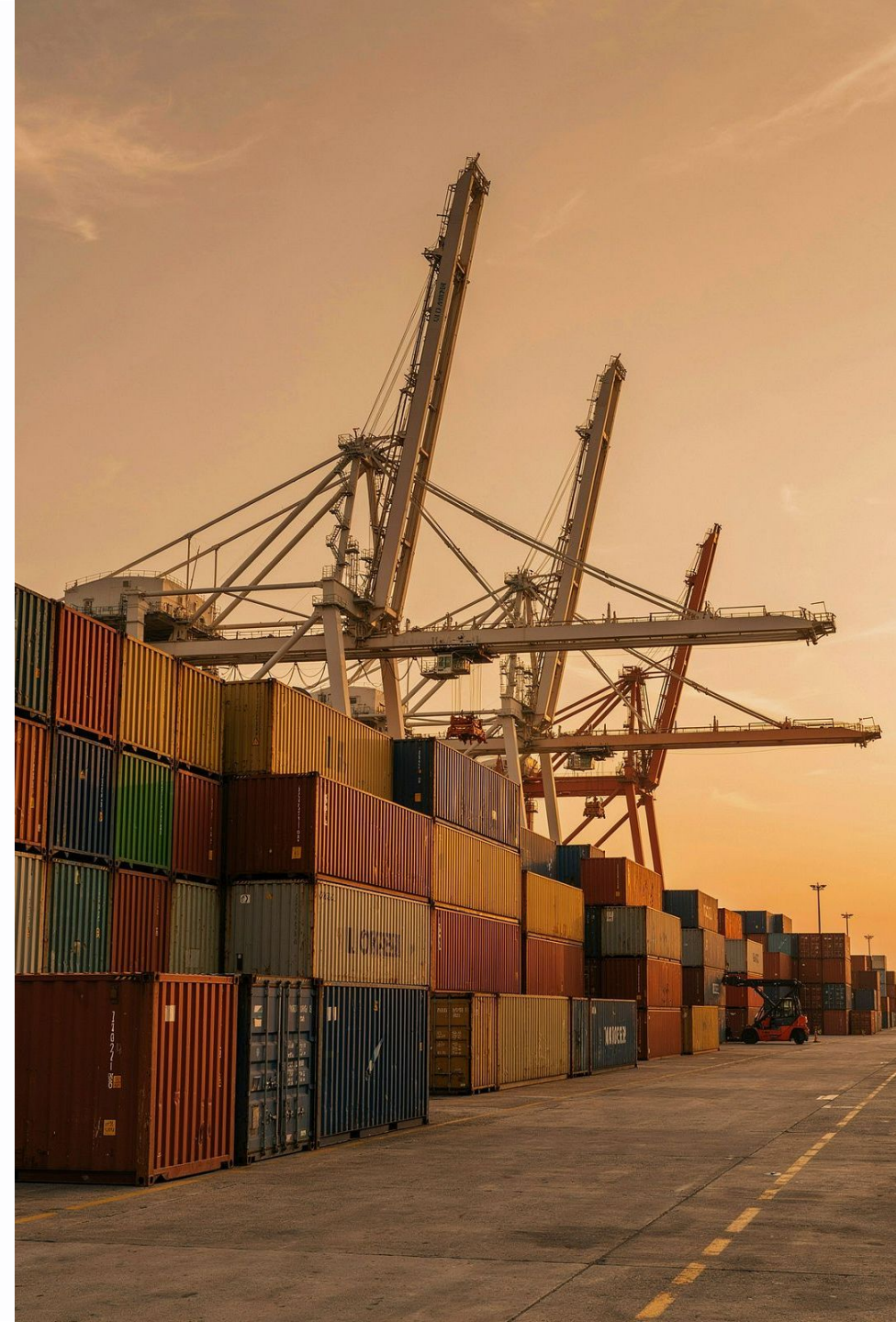
→ API Gateway & Rate Limiting

An API gateway is the single entry point that enforces auth, logging, and throttling. Rate limiting prevents abuse – think of it as a bouncer with a clicker counter.

MODULE 5

Microservices Security

When a monolith breaks into dozens of microservices, the attack surface multiplies. Every service-to-service call is a potential vulnerability — and securing the mesh is as important as securing the perimeter.



Securing Service-to-Service Communication

In a microservices architecture, services talk to each other constantly. Without proper controls, a compromised service can impersonate others, intercept traffic, or steal secrets.



Mutual TLS (mTLS)

Both sides of a connection present certificates. Unlike regular TLS, the server also verifies the client's identity. Real-world analogy: both parties show ID before a conversation — not just the visitor, but also the receptionist.



Secret Management

Never hardcode credentials. Use dedicated secret stores (HashiCorp Vault, AWS Secrets Manager) to inject secrets at runtime. Rotate secrets automatically and audit every access event.



Service Mesh

A dedicated infrastructure layer (e.g., Istio, Linkerd) that handles mTLS, observability, and traffic policies automatically — without changing application code. Think of it as a security layer injected at the network level.



Supply Chain Security

Verify every dependency — a malicious open-source package can compromise your entire microservice fleet. Use SBOM (Software Bill of Materials), container image signing, and dependency scanning in your CI/CD pipeline.

Secure Configuration in Microservices

Misconfiguration is the #1 cause of cloud breaches. In microservices environments, configuration sprawl makes this risk even higher — hundreds of services, each with its own settings.

What Can Go Wrong

- Debug endpoints left enabled in production
- Default credentials on service-to-service calls
- Overly permissive IAM roles attached to services
- Secrets stored in environment variables instead of a vault
- Unrestricted egress allowing data exfiltration

Best Practices

- Apply least privilege IAM roles per service — not shared roles
- Use infrastructure-as-code (IaC) with security linting (e.g., Checkov, tfsec)
- Enforce policy-as-code with tools like OPA (Open Policy Agent)
- Scan container images for known CVEs before deployment
- Implement configuration drift detection — alert when prod diverges from baseline

📄 Treat your configuration as code: version-controlled, reviewed, and tested before deployment.

MODULE 6

Encryption

Encryption is your last line of defense — even when all other controls fail, properly encrypted data is useless to an attacker. Understanding when and how to apply it is a core cloud security skill.



Encryption at Rest, in Transit & Key Management

Encryption at Rest

Protects data stored on disk, in databases, or in object storage. If an attacker gets a copy of your S3 bucket or database volume, encrypted data is unreadable without the key. Use AES-256 as standard. Enable default encryption on all cloud storage resources.

Encryption in Transit

Protects data moving across networks using TLS 1.2 or 1.3. Enforce HTTPS everywhere — even on internal service-to-service calls. Prevent protocol downgrades and validate certificates to avoid man-in-the-middle attacks. Real-world analogy: sending a letter in a locked box, not a postcard.

TLS & Certificates

TLS (Transport Layer Security) is the protocol powering HTTPS. Certificates — issued by a Certificate Authority — prove a server's identity. Automate certificate rotation (e.g., with Let's Encrypt or AWS ACM) to avoid expiry-caused outages.

Key Management & Cloud KMS

Encryption is only as strong as your key management. Cloud KMS (AWS KMS, Google Cloud KMS, Azure Key Vault) provides:

- Centralized key generation, rotation, and auditing
- Hardware Security Module (HSM) backing for highest assurance
- Fine-grained IAM policies on who can use which key

❏ Rotate encryption keys regularly. A key that never rotates is a key that will eventually be compromised.

Key Takeaways — Day 2

Today's six modules form an interconnected security framework. Each principle reinforces the next — strong design principles make threat modeling more effective, which makes your API and microservice defenses more targeted, all protected by solid encryption.

Design for Security

CIA Triad, Least Privilege, Zero Trust, and Defense in Depth are the non-negotiable foundation.

Model Before You Build

Use STRIDE and DFDs to find flaws in design — not in production after a breach.

Secure Every API

Authenticate, authorize, validate, and rate-limit every request. Trust nothing by default.

Harden the Mesh

mTLS, secret management, and supply chain hygiene are table stakes for microservices.

Encrypt Everything

At rest, in transit, and manage keys centrally. Never hardcode credentials or skip rotation.